

Annales des examens

[PRÉCÉDENT](#) | [INDEX](#)

Annales des examens

Examen blanc, janvier 2019

Le schéma de base de données suivant sera utilisé pour l'ensemble du sujet. Il permet de gérer une plateforme de vente de billets de spectacles en ligne :

- `Personne(id, nom, prénom, age)`
- `Salle(id, nom, adresse, nbPlaces)`
- `Spectacle(id, idArtiste, idSalle, dateSpectacle, nbBilletsVendus)`
- `Billet(id, idSpectateur, idSpectacle, catégorie, prix)`

Notez que les artistes et les spectateurs sont des personnes. Tous les attributs **id** sont des clés primaires, ceux commençant par **id** sont des clés étrangères.

Conception (5 points)

- Donnez le modèle entité / association correspondant au schéma de notre base.
- Pensez-vous qu'une association a été réifiée ? Si oui, laquelle ?
- Donnez la commande de création de la table `Spectacle`
- On veut qu'un même spectateur puisse avoir au plus un billet pour un spectacle. Quelle dépendance fonctionnelle cela ajoute-t-il ? Que proposez-vous pour en tenir compte dans le schéma ?

SQL (6 points)

Question de cours. Deux requêtes syntaxiquement différentes sont équivalentes si

- Elles peuvent donner le même résultat
- Elles donnent toujours le même résultat, quelle que soit la base

Indiquez la bonne réponse.

Donnez les expressions SQL des requêtes suivantes

- Noms des artistes âgés d'au moins 70 ans qui ont donné au moins un spectacle dans la salle "Bercy"
- Donnez le nom des artistes, des salles où ils se sont produits, triés sur le nombre de billets vendus
- Quelles personnes n'ont jamais acheté de billet?
- Donnez la liste des spectateurs ayant acheté au moins 10 billets, affichez le nom, le nombre de billets et le prix total.

Algèbre (5 points)

Voici 3 relations.

A B C D

1 0 1 2
4 1 2 2
6 2 6 3
7 1 1 3
1 0 1 1
1 1 1 1

Relation R



Ce cours de [Philippe Rigaux](#) est mis à disposition selon les termes de la [licence Creative Commons Attribution Pas d'Utilisation Commerciale - Partagée dans les Mêmes Conditions 4.0 International](#).

TABLE OF CONTENTS

Introduction
Le modèle relationnel
SQL, langage déclaratif
SQL, langage algébrique
SQL, récapitulatif
Conception d'une base de données
Schémas relationnel
Procédures et déclencheurs
Transactions
Une étude de cas
Annales des examens
Examen blanc, janvier 2019
Conception (5 points)
SQL (6 points)
Algèbre (5 points)
Transactions (4 points)
Correction de l'examen
Examen session 2, avril 2019
Conception (6 points)
SQL (8 points)
Algèbre (3 points)
Programmation et transaction points)
Corrigé
Examen session 1, présentiel, juin 2019
Conception (6 points)
SQL (8 points)
Algèbre (3 points)
Valeurs nulles (2 pts)
Transactions (1 pt)
Examen session 2, présentiel, septembre 2020
Schéma relationnel (6 points)
SQL (7 points)
Un peu de logique (3 points)
Algèbre (2 points)
Transactions (2 points)
Examen session 1, FOAD, janvier 2022
Conception (8 points)

ABE

1 0 2

4 1 2

1 0 7

8 6 6

Relation S

ABE

1 0 2

4 2 2

1 0 7

8 6 5

8 5 6

Relation T

- Pour quelle requête le résultat contient-il plus d'un nuplet? Attention: souvenez-vous que l'opérateur de projection élimine les doublons.

- $\pi_{A,C}(\sigma_{B=0}(R))$
- $\pi_{A,C}(\sigma_{D=0}(R))$
- $\pi_{A,C}(\sigma_{B=0}(R) \cup \sigma_{D=0}(R))$
- $\pi_{A,C}(\sigma_{A=C}(R))$

- Combien de nuplets retourne la requête $\pi_{A,B,E}(S \bowtie_{A=A \wedge B=B} R)$?

- 2
- 3
- 4
- 5

- Combien de nuplets retourne la requête $R \bowtie_{A=A \wedge B=B} (S \cup T)$?

- 3
- 5
- 7
- 9

- Donnez l'expression algébrique pour la requête « Noms et prénoms des spectateurs qui ont acheté au moins une fois un billet de plus de 500 euros. »
- Donnez l'expression algébrique pour la requête « Id des personnes qui ne sont pas des artistes »

Transactions (4 points)

On prend le schéma donné au début de l'examen.

- Question de cours.** Quel est le principal inconvénient du mode `serializable`?

- Le système ne peut exécuter qu'une transaction à la fois
- Certaines transactions sont mises en attente avant de pouvoir finir
- Certaines transactions sont rejetées par le système

Indiquez la bonne réponse

- Exprimez au moins un critère de cohérence de cette base
- On écrit une procédure de réservation d'un billet pour un spectacle.

```
Reserver (v_idSpectateur, v_idSpectacle, v_categorie,v_ prix)
# Requête SQL de mise à jour du spectacle
requêteA
```

SQL (7 points)

Algèbre relationnelle (3 pts)

Jointure externe (2 pts)

Examen session 1, second semestre 2023

Conception du schéma (7 pts)

SQL (7 pts)

Algèbre (2 pts)

Valeurs nulles, vues (2 pts)

Procédures (2 pts)

Examen session 1, 27 janvier 2025

Modélisation (8 points)

Requêtes SQL (7 points)

Algèbre (2 points)

Programmation (3 points)

Examen session 1, 12 juin 2025

Compréhension du schéma (6 pts)

Algèbre et vues (4 pts)

SQL (7 pts)

Modèle relationnel (3 pts)

RECHERCHE

Saisissez un ou plusieurs mots-clés.

```
# Requête SQL d'insertion du billet
requêteB
```

Donnez les deux requêtes SQL, indiquez l'emplacement du `commit` et justifiez-les brièvement.

Correction de l'examen

Conception

Le schéma E/A est donné dans la [Fig. 73](#). Notez que l'on ne représente pas les clés étrangères comme attributs des entités: les clés étrangères sont le mécanisme *du modèle relationnel* pour représenter les liens entre entités. Dans le *modele EA*, ces liens sont des associations, et il serait donc redondant de faire figure également les clés étrangères

Le nommage des associations et des attributs est libre, l'important est de privilégier la clarté et la précision, notamment pour les associations.

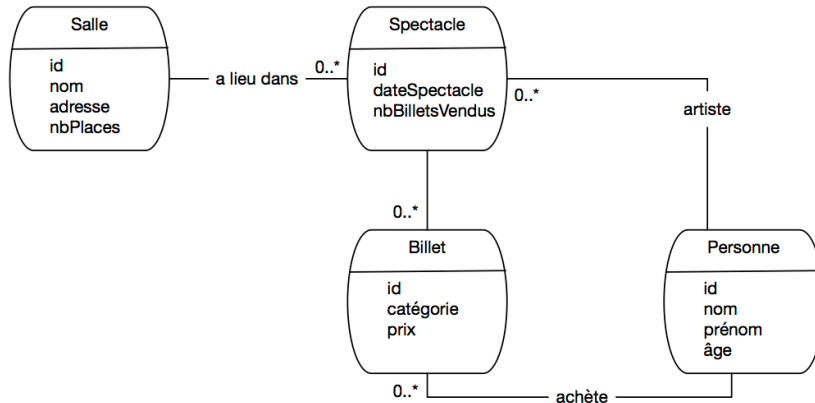


Fig. 73 Le schéma E/A après rétro-conception

Le type d'entité **Billet** est issu d'une réification d'une association plusieurs-plusieurs. La principale différence est que les billets ont maintenant leur identifiant propre, alors que l'association plusieurs-plusieurs est identifiée par la paire (idSpectacle, idPersonne).

Voici la commande de création de la salle. On a choisi de mettre `not null` partout pour maximiser la qualité de la base, ce sont des contraintes que l'on peut facilement lever si nécessaire.

```
create table Spectacle (id integer not null,
    idArtiste integer not null,
    idSalle integer not null,
    dateSpectacle date not null,
    nbBilletsVendus integer not null,
    primary key (id),
    foreign key (idArtiste) references Personne (id),
    foreign key (idSalle) references Salle (id)
)
```

Dans le schéma relationnel proposé, il est possible d'avoir plusieurs billets (avec des identifiants distincts) pour le même spectacle et le même spectateur. C'est un effet de la réification: on a perdu la contrainte d'unicité sur la paire (idSpectacle, idPersonne).

On peut se poser la question d'ajouter ou non cette contrainte. Si oui, alors (idSpectacle, idSpectateur) devient une clé secondaire, que l'on déclare avec la clause `unique` dans la commande de création de la table **Billet**.

```
create table Billet (id integer not null,
    idSpectacle integer not null,
    idSpectateur integer not null,
    catégorie varchar(4) not null,
    prix float not null,
    primary key (id),
    unique (idSpectacle, idSpectateur),
    foreign key (idSpectacle) references Spectacle (id),
    foreign key (idSpectateur) references Personne (id)
)
```

SQL

Deux requêtes sont équivalentes si elles donnent toujours le même résultat, quelle que soit la base.

```
select p.nom
from Personne as p, Spectacle s, Salle l
where p.id = s.idArtiste and s.idSalle = l.id
and l.nom = 'Bercy' and p.age >= 70
```

```
select p.nom as nomArtiste, s.nom as nomSalle, sp.nbBilletsVendus
from Personne as p, Spectacle as sp, Salle as s
where p.id = sp.idArtiste
and sp.idSalle = s.id
order by sp.nbBilletsVendus
```

```
select prénom, nom
from Personne as p
where not exists (select ' ' from Billet where idSpectateur = p.id)
```

```
select p.nom, count(*), sum(prix)
from Personne as p, Billet as b
where p.id = b.idSpectateur
group by p.id, p.nom
having count(*) >= 10
```

Algèbre

- La requête 4 renvoie 2 nuplets: (1,1) et (6,6)
- Voici l'union $S \cup T$:

ABE

1 0 2

4 1 2

8 6 6

4 2 2

1 0 7

8 6 5

8 5 6

La jointure sur A et B avec R renvoie 5 nuplets: (1,0,1,2) s'associe à 2 nuplets dans $S \cup T$, de même que (1,0,1,1). (4,1,2,2) s'associe à un nuplet.

Relation T

- La requête renvoie également 3 nuplets
- $\pi_{nom,prenom}(Personne \bowtie_{id=idSpectateur} \sigma_{prix>500}(BILLET))$
- $\pi_{id}(Personne) - \pi_{idArtiste}(Spectacle)$

Transactions

Le mode sérialisable entraîne des rejets de transactions.

Le nombre de billets vendus (nbBilletsVendus dans Spectacle) pour un spectacle est égal au nombre de lignes dans Billet pour ce même spectacle.

Autre possibilité: pas plus de billets vendus que de places dans la salle.

Un update de Spectacle pour incrémenter nbBilletsVendus,

```
update Spectacle set nbBilletsVendus=nbBilletsVendus+1 where idSpectacle=v_idSpectacle
```

et un insert dans Billet.

```
insert into Billet (id, idSpectateur, idSpectacle, catégorie, prix)
value (1234, v_idSpectateur, v_idSpectacle, v_catégorie, v_prix)
```

Le commit vient à la fin car c'est seulement à ce moment que la base est à nouveau dans un état cohérent.

Examen session 2, avril 2019

Soit la base de données suivante permettant de gérer un championnat de football.

- `Stade(id, ville, nom, nbPlaces, prixBillet)`
- `Equipe(id, pays, siteWeb, entraîneur)`
- `Joueur(id, idEquipe, nom, prénom, âge)`
- `Match(id, idStade, dateMatch, idEquipe1, idEquipe2, scoreEquipe1, scoreEquipe2, nbBilletsVendus)`
- `But(idJoueur, idMatch, minute, penalty)`

Tous les attributs **en gras** sont des clés primaires, ceux *en italiques* sont des clés étrangères. Notez qu'une clé étrangère peut faire partie d'une clé primaire.

Chaque pays a une seule équipe en compétition. Une ville peut avoir plusieurs stades. L'attribut `But.penalty` vaut VRAI si le but a été marqué suite à un penalty (coup de pied de réparation) ou FAUX dans le cas contraire. Le nom de l'entraîneur d'une équipe est donné par l'attribut `Equipe(entraîneur)`.

Conception (6 points)

Étudions la conception de cette base en répondant aux questions suivantes.

- Avec ce schéma, une équipe peut-elle jouer contre elle-même? Expliquez.
- Avec ce schéma, un joueur peut-il marquer un but dans un match auquel son équipe ne participe pas? Expliquez.
- Est-il possible de savoir dans quelle ville a été marqué chaque but? Expliquez.
- Donnez le schéma entité-association correspondant aux relations `Match`, `Equipe`, `Stade`.
- Donnez la commande SQL de création de la table `Match`.
- Citer au moins une clé *candidate* autre que la clé primaire parmi les attributs des tables (bien lire l'énoncé). Quel serait l'impact si on la choisissait comme clé primaire?
- Un de vos collègues propose de modéliser un but comme une association plusieurs-plusieurs entre un joueur et un match. Cela vous semble-t-il une bonne idée? Cela correspond-il au schéma relationnel de notre base?

SQL (8 points)

Exprimez les requêtes suivantes en SQL.

- Noms des joueurs âgés de plus de 30 ans qui ont marqué un but dans la première minute de jeu.
- Noms des joueurs français qui n'ont marqué aucun but
- Le prix du billet le plus cher
- Donnez le nom des entraîneurs avec le nombre de buts marqués par leur équipe
- Donnez le nom des joueurs ayant marqué un but dans un match auquel leur équipe ne participe pas.
- Les noms des joueurs français qui ont marqué un but lors d'un match entre la France et la Suisse joué à Lille. Attention: dans la table `Match` la France peut être l'équipe 1 ou l'équipe 2. Il faut construire la requête qui correspond aux deux cas.

Algèbre (3 points)

- Donnez l'expression algébrique de la première requête SQL (section précédente)
- Donnez l'expression algébrique pour la seconde requête (les joueurs français qui ne marquent pas)
- Donnez la requête SQL correspondant à l'expression algébrique suivante et expliquez le sens de cette requête

$$\pi_{idMatch}(But \bowtie_{idMatch=id} \sigma_{scoreEquipe1=0 \wedge scoreEquipe2=0}(Match))$$

Programmation et transactions (3 points)

- Parmi les phrases suivantes, lesquelles vous semblent exprimer une contrainte de *cohérence* correcte sur la base de données?

- Pour chaque match nul, je dois trouver autant de lignes associées au match dans la table But pour l'équipe 1 et pour l'équipe 2.
- Quand j'insère une ligne dans la table But pour l'équipe 1, je dois effectuer également une insertion pour l'équipe 2 dans la table But.
- Si je trouve une ligne dans la table Match avec $id=x$, $scoreEquipe1=y$ et $scoreEquipe2=z$, alors je dois trouver $y+z$ lignes dans la table But avec $idMatch=x$.
- Voici un programme en pseudo-code simplifié à exécuter chaque fois qu'un but est marqué dans un match, par un joueur et au bénéfice de la première équipe. Les `"#"` marquent des commentaires.

```
function marquer_but_eq1 (imatch, ijoueur, min, peno)
  startTransaction
  # A
  select scoreEquipe1 into :scoreE where idMatch = :imatch
  #B
  update Match set scoreEquipe1 = :scoreE + 1 where idMatch = :imatch
  # C
  insert into But (idMatch, idJoueur, minute, penalty)
  values (:idMatch, :ijoueur, :min, :peno)
  # D
```

Où faut-il selon-vous ajouter un ordre commit?

- Juste après B et C
- Juste après A
- Juste après D

Justifiez votre réponse.

Supposons que je lance deux exécutions de `marquer_but_eq1` en même temps. Dans quel scénario l'exécution imbriquée amène-t-elle une incohérence?

[Corrigé](#)

[Conception](#)

- Oui, la base permet de représenter un but marqué dans un match opposant deux équipes dont aucune n'est celle du joueur.
- Oui, par transitivité $But \rightarrow Match$, $Match \rightarrow Stade$ et $Stade \rightarrow Ville$.
- Schéma

```
CREATE TABLE Match(
  id NUMBER(10),
  idStade NUMBER(10),
  dateMatch DATE,
  idEquipe1 NUMBER(10),
  idEquipe2 NUMBER(10),
  scoreEquipe1 NUMBER(10),
  scoreEquipe2 NUMBER(10),
  PRIMARY key (id),
  foreign key (idStade) REFERENCES Stade(id),
  foreign key (idEquipe1) REFERENCES Equipe(id),
  foreign key (idEquipe2) REFERENCES Equipe(id)
);
```

- Le pays est clé candidate pour l'équipe. Il faudrait alors l'utiliser comme clé étrangère partout.

[SQL](#)

```
select nom
from Joueur as j, But as b
where j.id = b.idJoueur and j.age > 30 and b.minute = 1;

select nom
from Joueur as j, Equipe as e
where j.idEquipe = e.id
```

```

and e.pays = 'France'
and j.id not IN (select idJoueur from But);

select prixBillet
from Stade as s1
where not exists (select ''
                  from Stade as s2
                  where s2.prixBillet > s1.prixBillet);

select entraineur, count(*)
from Equipe as e, Joueur as j, But as b
where e.id=j.idEquipe
and b.idJoueur = j.id
group by e.id, e.entraineur;

select prénom, nom
from Match as m, Joueur as j, But as b
where e.id=j.idEquipe
and b.idJoueur = j.id
and m.id = b.idMatch
and e.id != m.idEquipe1
and e.id != e.idEquipe2

```

Il faut soit Match.equipe1="France", soit Match.equipe2="France".

```

select j.nom
from Match m, Equipe e1, Equipe e2, Joueur j, Stade s, But b
where m.idEquipe1 = e1.id
and m.idEquipe2 = e2
and ((e1.pays = 'France' id and e2.pays='Suisse' and e1.id = j.idEquipe)
or
(e2.pays = 'France' id and e1.pays='Suisse' and e2.id = j.idEquipe))
and m.idStade = s.id
and s.ville = 'Lille'
and b.idJoueur = j.id
and b.idMatch = m.id )

```

Algèbre

$$\pi_{nom}(\sigma_{age>30}(Joueur) \bowtie_{id=idJoueur} \sigma_{minute=1}(But))$$

Joueurs français:

$$A = \sigma_{pays='France'}(Joueur \bowtie_{j.idEquipe=e.id} Equipe)$$

Joueurs français qui ont marqué au moins un but :

$$B = \sigma_{pays='France'}(But \bowtie_{b.idJoueur=j.id} Joueur \bowtie_{j.idEquipe=e.id} Equipe)$$

Résultat:

$$Resultat := A - B$$

Les matchs nuls 0-0 pour lequel il existe quand même un but.

```

select idMatch
from But as b, Match as m
where b.idMatch=m.id
and m.scoreEquipe1=0
and m.scoreEquipe2=0

```

Examen session 1, présentiel, juillet 2019

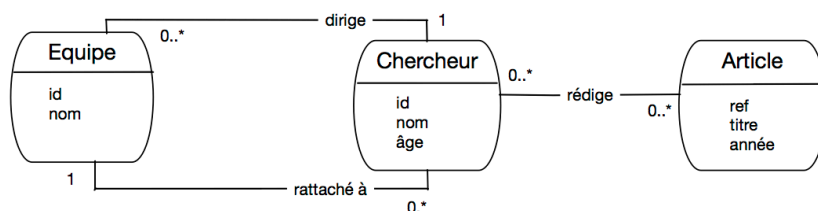


Fig. 74 Un laboratoire d'informatique

On cherche à modéliser un laboratoire d'informatique afin de pouvoir gérer ses équipes, ses chercheurs et leurs publications (articles). L'analyse donne le schéma entité/association de la [Fig. 74](#). À partir de cette analyse, quelqu'un propose le schéma relationnel suivant, dans lequel les clés primaires sont en gras (à vous de trouver les clés étrangères):

- Equipe (**id**, nom, idDirecteur)
- Chercheur (**id**, nom, âge, idEquipe)
- Article (**réf**, titre, année)
- Rédige (**idChercheur**, réfArticle)

Conception (6 points)

- Ce schéma relationnel représente-t-il correctement le modèle conceptuel de la [Fig. 74](#)? Vérifiez les clés primaires et étrangères et proposez éventuellement des corrections.
- On veut ajouter la position d'un chercheur dans l'ordre des auteurs pour un article. Où placer cette information, dans le modèle entité association et dans le modèle relationnel?
- Un article peut-il être rédigé par des chercheurs appartenant à des équipes différentes (justifiez)?
- Un chercheur peut-il diriger une équipe à laquelle il n'est pas rattaché (justifiez)?
- Donnez les commandes `create table` pour les tables Chercheur, Article et Rédige (en tenant compte des corrections éventuelles de la question 1).

Correction

- La clé de Rédige doit comprendre réfArticle en plus de idAuteur
- On ajoute la position dans l'association Rédige (et dans la table correspondante).
- Oui, pas de restriction dans le schéma
- Oui, pas de restriction dans le schéma
- Standard. Exemple pour la relation Rédige:

```
create table Redige (idAuteur    int not null,
                    réfArticle  varchar (10) not null,
                    primary key (idAuteur, réfArticle),
                    foreign key (idAuteur) references Chercheur(id),
                    foreign key (réfArticle) references Article(id));
```

SQL (8 points)

Exprimez les requêtes suivantes en SQL.

- Noms des chercheurs de l'équipe nommée Vertigo
- Titre des articles parus depuis 2015 dont l'un au moins des auteurs appartient à l'équipe ROC
- Quels articles n'ont pas d'auteur?
- Titre des articles parus depuis 2015 dont tous les auteurs appartiennent à l'équipe ROC (aide: la quantification universelle se remplace par la quantification existentielle et la négation)
- Nom des chercheurs qui dirigent une équipe à laquelle ils n'appartiennent pas
- Nom des chercheurs qui dirigent plus d'une équipe.

Correction

```
select c.nom as nomChercheur
from Chercheur as c, Equipe as e
where e.nom='Vertigo' and c.idEquipe=e.id
```

```
select distinct titre
from article as a, chercheur as c, equipe as e, redige as r
where a.ref = r.efarticle
and r.idchercheur=c.id
```



```

and c.idequipe=e.id
and a.annee >= 2015
and e.nom = 'ROC '

select a.titre
from Article as a
where no exists (select 'x'
                 from Rédige where réfArticle=a.réf)

select a.titre
from Article as a
where a.réf not in (select réfArticle
                  from Rédige)

select titre
from article as a
where a.annee >= 2015
and not exists (select '' from Redige as r, Chercheur as c, Equipe as e
                where r.refArticle = a.ref and c.id=r.idAuteur
                and e.id=c.idEquipe and not (e.nom = 'ROC'))

select c.nom as nomChercheur
from Chercheur as c, Equipe as e
where e.idDirecteur = c.id
and e.id <> c.idequipe

select c.id as idChercheur, c.nom as nomChercheur, count(*) as nbEquipes
from chercheur as c, equipe as e
where e.idDirecteur=c.id
group by c.id, c.nom
having count(*) >1

select e.idDirecteur, count(*) as nbEquipes
from Equipe as e
group by e.idDirecteur
having count(*) >1

```

Algèbre (3 points)

- Donnez l'expression algébrique pour les deux premières requêtes SQL (section précédente)
- Donnez la requête SQL correspondant à l'expression algébrique suivante et expliquez-en le sens.

$$\pi_{id}(Chercheur) - \pi_{idChercheur}(Rédige)$$

Correction

- $\pi_{nom}(\pi_{id}(\sigma_{nom='Vertigo'}(Equipe)) \bowtie_{id=idEquipe} Chercheur)$
- $\pi_{titre}(((\sigma_{annee \geq 2015}(Article) \bowtie_{ref=refArticle} Rédige) \bowtie_{idChercheur=id} Chercheur) \bowtie_{idEquipe=id} \sigma_{nom='ROC'}Equipe)$
- L'expression recherche les chercheurs qui n'ont rien publié.

```

select id
from Chercheur
where id not in (select idChercheur from Rédige)

```

Valeurs nulles (2 pts)

Le tableau suivant montre une instance de Article. Les cellules blanches indiquent des valeurs inconnues.

réf	titre	année
AR243	Les ordinateurs pensent-ils?	2018
AR254	Money for nothing	
AR20		2010

Donnez le résultat des requêtes suivantes:

```
select réf from Article where année > 2015;
```

```
select réf from Article where année <= 2015;
```

```
select réf from Article where annee > 0 or titre is not null;
```

```
select réf from Article where (année < 2015 or année IS null)
and titre like `_%`
```

Correction

- AR243
- AR20
- AR243, AR254, AR20
- AR254

Transactions (1 pt)

Soit deux transactions concurrentes T_1 et T_2 . Quelle(s) affirmation(s), parmi les suivantes, est (sont) vraie(s) dans un système transactionnel assurant les propriétés ACID en isolation complète.

- Si T_2 débute après T_1 , T_1 ne voit jamais les mises à jour de T_2 .
- T_1 ne voit pas ses propres mises à jour tant qu'elle n'a pas validé.
- Si T_2 débute après T_1 , T_1 ne voit les mises à jour de T_2 qu'après que T_2 a effectué un commit.
- Si T_1 et T_2 veulent modifier le même nuplet, cela déclenche un interblocage (deadlock).

Correction

Seule la première affirmation est vraie

Examen session 2, présentiel, septembre 2020

Voici une partie de la base de données utilisée pour gérer des œuvres dans un musée:

- Oeuvre(**idOeuvre**, nom, idPropriétaire, idAuteur)
- Personne(**idPersonne**, nom, prénom)
- Expertise(**idExpertise**, idOeuvre, idExpert, valeur)

Les auteurs, les propriétaires et les experts sont des personnes bien entendu. Les clés primaires sont en gras, les clés étrangères ne sont pas explicitement indiquées.

Schéma relationnel (6 points)

- Pour chaque table, donner les dépendances fonctionnelles liant les clés primaires et clés étrangères (1 point)
- En déduire les associations de type *plusieurs à 1* du schéma Entité/Association correspondant à ce schéma relationnel (1 point)
- Donner ce schéma Entité/Association complet (1 point)
- Donner des commandes SQL permettant de créer les tables Oeuvre et Personne (1 point)
- J'ajoute la contrainte *Un expert ne peut évaluer une œuvre qu'une seule fois*. Quelle est la dépendance fonctionnelle correspondante et quel changement du schéma relationnel proposez-vous? (1 point)
- Ce schéma permet-il de représenter le fait qu'une personne peut être à la fois propriétaire et auteur d'une même œuvre? Argumentez votre réponse. (1 point)

Correction

1. Dépendances entre clés primaires et clés étrangères :

- Dans *Oeuvre*: $idOeuvre \rightarrow idAuteur$ et $idOeuvre \rightarrow idProprietaire$
- Dans *Expertise*: $idExpertise \rightarrow idOeuvre$ et $idExpertise \rightarrow idExpert$

1. Chaque DF correspond à une association plusieurs à 1 (cf l'algorithme de normalisation). On a donc ce type d'association entre *Oeuvre* et *Personne*, avec la sémantique « Auteur », entre *Oeuvre* et *Personne* une seconde fois mais avec la sémantique « Propriétaire », entre *Expertise* et *Oeuvre* et entre *Expertise* et *Personne*.
2. Le schéma EA se déduit immédiatement de ce qui précède. Il n'y a pas d'association plusieurs à plusieurs.

```
3. CREATE TABLE Personne (
    idPersonne INTEGER NOT NULL,
    nom VARCHAR NOT NULL,
    prénom VARCHAR NOT NULL,
    PRIMARY KEY (idPersonne)
);

CREATE TABLE Oeuvre (
    idOeuvre INTEGER NOT NULL,
    nom VARCHAR,
    idAuteur INTEGER NOT NULL,
    idProprietaire INTEGER NOT NULL,
    PRIMARY KEY (idOeuvre),
    FOREIGN KEY (idAuteur) REFERENCES Personne (idPersonne)
    FOREIGN KEY (idProprietaire) REFERENCES Personne (idPersonne)
);
```

4. On a donc la nouvelle dépendance $(idExpert, idOeuvre) \rightarrow valeur$. Du coup la table *Expertise* n'est plus en 3FN. Un changement possible est que $(idExpert, idOeuvre)$ devienne la clé de *Expertise*.
5. Oui, rien n'empêche $idAuteur$ et $idProprietaire$ d'être égaux dans la table *Oeuvre*

SQL (7 points)

- Donner le nom du propriétaire et le nom de l'auteur pour l'œuvre dont l'identifiant est "fg65".
- Donner les noms des œuvres expertisées par leur propriétaire
- Donner les noms et prénoms des propriétaires d'une œuvre qui n'ont jamais effectué d'expertise.
- Donner les œuvres et le nom des personnes qui n'en sont ni propriétaire, ni auteur.
- Donner les noms et prénoms des personnes qui sont à la fois auteur, propriétaire et expert (mais pas forcément de la même œuvre).
- Pour chaque œuvre donnez la moyenne des valeurs estimées par les experts.
- Donnez le nombre d'expertises pour les œuvres dont la valeur maximale estimée est de 10 000 Euros

Correction

```
select p1.nom as nomAuteur, p2.nom as nomProprietaire
from Oeuvre as o, Personne as p1, Personne as p2
where idOeuvre='fg65'
and p.idAuteur=p1.idPersonne
and p.idProprietaire = p2.idPersonne;

select p1.nom as nomAuteur, p2.nom as nomProprietaire
from Oeuvre as o, Expertise as e
where o.idOeuvre=e.idOeuvre
and o.idProprietaire = e.idPersonne

select p.nom, p.prénom
from Oeuvre as o, Personne as p
where p.idProprietaire = p.idPersonne
and p.idPersonne not in (select idExpert from Expertise)

select o.nom as nomOeuvre, p.nom as nomPersonne
from Oeuvre as o, Personne as p
```

```

where o.idPropriétaire != p.idPersonne
and o.idAuteur != p.idPersonne

select distinct p.nom
from Personne as p, Oeuvre as o1, Oeuvre as o2, Expertise as e
where o1.idPropriétaire = p.idPersonne
and o2.idAuteur = p.idPersonne
and e.idExpert = p.idPersonne

select o.nom, avg(valeur)
from Oeuvre as o, Expertise as e
where o.idOeuvre = e.idOeuvre
group by o.idOeuvre, o.nom

select o.nom, count(*)
from Oeuvre as o, Expertise as e
where o.idOeuvre = e.idOeuvre
group by o.idOeuvre, o.nom
having max(valeur) <= 10000

```

Un peu de logique (3 points)

On dispose de deux prédicats $Auteur(x)$ et $Prop(x)$ qui sont vrais si, respectivement, x est auteur ou x est propriétaire.

- Quelle formule exprime la condition « Soit x est propriétaire, soit x est auteur mais pas les deux »
- Quelle est la négation de l'énoncé : « Soit x est propriétaire, soit x est auteur »
- Comment exprimer l'énoncé suivant sans implication: « Si x est auteur, alors x n'est pas propriétaire » (utiliser uniquement les connecteurs de SQL: and, or et not).

Correction

- $Auteur(x) \vee Prop(x) \wedge \neg(Auteur(x) \wedge Prop(x))$
- $\neg Auteur(x) \wedge \neg Prop(x)$
- L'implication : math : est équivalente à $\neg p \vee q$. Donc en SQL on écrirait cette formule : $not(Auteur(x)) \vee Prop(x)$

Algèbre (2 points)

On dispose de deux tables $T_1(A, B, C)$ et $T_2(D, E, F)$. Donnez une expression algébrique équivalente à la suivante, avec les contraintes suivantes: on peut utiliser la jointure mais pas le produit cartésien, et une sélection doit s'appliquer directement à une table.

$$\sigma_{A=C \wedge A=D \wedge E=F \wedge C > B}(T_1 \times T_2)$$

Correction

$$\sigma_{A=C \wedge C > B}(T_1) \bowtie_{A=D} \sigma_{E=F}(T_2)$$

Transactions (2 points)

On dispose d'une table $T(id, valeur)$. Initialement toutes les valeurs sont différentes. Voici une procédure qui échange les valeurs de 2 nuplets.

```

create or replace procedure Echange (id1 INT, id2 INT) AS
-- Déclaration des variables
val1, val2 integer;
begin
-- On recherche la valeur de id1 et de id2
select valeur into val1 from T where id = id1
select valeur into val2 from T where id = id2

-- On échange les valeurs
update T SET valeur = val1 where id = id2
update T SET valeur = val2 where id = id1

end;

```

On est en mode Autocommit: un commit a lieu après chaque requête SQL. Expliquez dans quel scénario l'exécution concurrente de deux procédures d'échange peut aboutir à ce que deux nuplets aient la même valeur.

Correction

La première procédure s'exécute jusqu'à la fin du premier UPDATE. Un commit automatique a lieu: les deux nuplets sont alors égaux. Si la seconde procédure commence alors, elle trouvera une base avec deux nuplets égaux, ce qui ne devrait jamais arriver avec nos hypothèses en mode sérialisable.

Examen session 1, FOAD, janvier 2022

Le schéma de base de données suivant sera utilisé pour l'ensemble du sujet. Il permet de gérer les souscriptions pour un ensemble d'opérateurs de téléphonie mobile. Ce schéma sera nommé **schéma final** par la suite.

- Opérateur (**id**, nom)
- Forfait (**id**, nom, idOpérateur, prix)
- Client(**id**, nom, prénom, ville)
- Souscription(**idClient**, **idForfait**, durée, numéro)

L'attribut *durée* est un entier positif exprimant le nombre de mois d'engagement. les clés étrangères ne sont pas indiquées.

Conception (8 points)

La [Fig. 75](#) montre une modélisation initiale de la base par un schéma entité association.

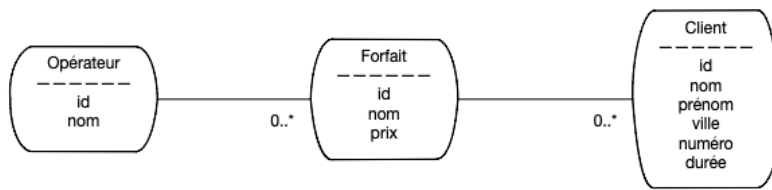


Fig. 75 Modélisation initiale de la base

Questions:

- Donnez la liste des dépendances fonctionnelles définies par ce schéma initial (1 pt)
- Donnez le schéma relationnel correspondant à la [Fig. 75](#), sous forme simplifiée (Nom de table, liste des attributs en encadrant les clés primaires et en soulignant les clés étrangères) (1 pt)
- Quelles sont les différences entre le schéma initial et le schéma final ? À quelle(s) évolution(s) des choix de modélisation ce changement correspond-t-il (2 pts) ?
- Dessinez le schéma final sous forme entité-association. Quelles dépendances fonctionnelles ont changé par rapport au schéma initial ? (1 pt)
- Dans le schéma final, une réification serait-elle possible? Quels changements impliquerait-elle ? (1 pt)?
- Donnez les commandes `create table` pour le schéma final (2 pts)

Correction

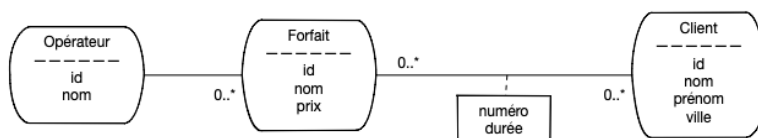


Fig. 76 Modélisation finale de la base

- Chaque entité définit une DF de la clé vers les attributs, et chaque association plusieurs-un définit une DF entre les clés. Donc : $idOpérateur \rightarrow nom$;
 $idForfait \rightarrow nom, prix, idOpérateur$;
 $idClient \rightarrow nom, prénom, ville, numéro, durée, idForfait$
- Schéma relationnel :

- Opérateur (**id**, nom)
- Forfait (**id**, nom, *idOpérateur*, prix)
- Client(**id**, nom, prénom, ville, durée, numéro, *idForfait*)
- Dans le schéma initial, un client peut prendre un seul forfait, d'où la DF $idClient \rightarrow idForfait$. Dans le schéma final un client peut prendre plusieurs forfaits (mais pas plusieurs fois le même), avec donc plusieurs numéros.
- Voir [Fig. 76](#). Une nouvelle clé est définie par l'association plusieurs-plusieurs, avec la DF $idClient, idForfait \rightarrow durée, numéro$
- En réifiant l'association plusieurs-plusieurs, on attribuerait un identifiant propre à la souscription, et un même client pourrait prendre plusieurs fois le même forfait, ce qui n'est pas possible dans le schéma final. Dans une « vraie » base, ce serait sans doute souhaitable.
- Classique. La table `Souscription` est typique d'une association plusieurs-plusieurs.

```
create table Souscription
(idClient integer not null,
 idForfait varchar not null,
 durée integer not null,
 numéro integer not null,
 primary key(idClient, idForfait),
 foreign key (idClient) references Client (id),
 foreign key (idForfait) references Forfait (id));
```

SQL (7 points)

Exprimez en SQL les requêtes suivantes **sur le schéma final, donné dans l'énoncé de l'examen**.

- Nom et prénom des clients qui ont souscrit un forfait avec engagement de plus de 24 mois.

Correction

```
select c.nom, c.prenom
from Client as c, Souscription as s
where s.idClient = c.id and s.duree > 24;
```

- Existe-t-il deux clients qui auraient le même numéro de téléphone pour le même forfait ? Donnez leurs noms et prénoms (des .clients)

Correction

```
select c1.nom as nom1, c1.prenom as prénom1, c2.nom as nom2, c2.prenom as prénom2
from Client as c1, Souscription as s1,
     Client as c2, Souscription as s2
where c1.id = s1.idClient,
and c2.id = s2.idClient
and s1.idForfait = s2.idForfait
and c1.numéro = c2.numéro
```

- Noms des clients qui ont un forfait nommé Audace et un autre nommé Privilège.

Correction

```
select nom, c.prenom
from Client as c, Souscription as s1, Souscription as s2,
     Forfait as f1, Forfait as f2
where c.id = s1.idClient,
and c.id = s2.idClient
and s1.idForfait = f1.idForfait
and s2.idForfait = f2.idForfait
and f1.nom='Audace' and f2.nom='Privilège'
```

- Quels clients n'ont pas souscrit de forfait ?

Correction

```
select c.nom, c.prenom
from Client as c
where not exists
  (select * from Souscription as s
   where c.id = s.idClient)
```

- Quels opérateurs n'ont pas de client à Lyon ?

Correction

```
select o.nom
from Opérateur as o
where not exists
  (select * from Forfait as f, Souscription as s, Client as c
   where o.id=f.idOpérateur
   and f.id=s.idForfait
   and s.idClient=c.id
   and c.ville='Lyon')
```

- Donnez le nombre de souscriptions pour chaque forfait de l'opérateur Violet.

Correction

```
select f.nom, count(*) as nbSouscriptions
from Opérateur as o, Forfait as f, Souscription as s
where o.id=f.idOpérateur
and f.id=s.idForfait
and o.nom='Violet'
group by f.id, f.nom
```

- Quels clients ont deux souscriptions ou plus ? Donnez le nombre de souscriptions.

Correction

```
select c.nom, c.prénom, count(*) as nbSouscriptions
from Client as c, Souscription as s
where c.id=s.idClient
group by c.id, c.nom
having count(*) > 1
```

Algèbre relationnelle (3 pts)

- Expliquez ce que fait la requête algébrique suivante, et donnez une expression SQL équivalente

$$\pi_{nom, prenom}(\sigma_{ville='Paris'}(Client) \bowtie_{id=idClient} (\sigma_{duree>24}(Souscription) \bowtie_{idForfait=id} \sigma_{nom='Audace'}(Forfait)))$$

- Même question pour l'expression suivante

$$\pi_{c.id, c.nom, c.prenom}(Client) - \pi_{c.id, c.nom, c.prenom}(Client \bowtie_{c.id=s.idClient} \sigma_{duree \geq 48}(Souscription))$$

Correction

Les clients qui n'ont pas de souscription supérieure à 48 mois.

- Voici une expression SQL « algébrique »

```
select c.nom, c.prénom
from (Client as c join Souscription as s on c.id=s.idClient)
join (select * from Forfait as f where nom= 'Audace')
on s.idForfait = f.id
```

Donnez une expression SQL « déclarative » équivalente, et sans requête imbriquée.

Correction

```
select c.nom, c.prénom
from Client as c, Souscription as s, Forfait as f
where c.id=s.idClient
and f.nom= 'Audace'
and s.idForfait = f.id
```

Jointure externe (2 pts)

Que renvoie la requête suivante ?

```
select c.nom, c.prénom, f.nom
from (Client as c left outer join (Souscription as s join Forfait as f
                                on s.idForfait=f.id)
     on c.id = s.idClient
where ville= 'Nantes')
```

Correction

On obtient tous les clients de Nantes, et leurs forfaits. Si un client n'a aucun forfait, une ligne est produite quand même, avec le nom du forfait à null.

Examen session 1, second semestre 2023

Une application met en contact des personnes dont certaines (les professeurs) proposent des leçons en ligne à d'autres (les étudiants). Dans ce qui suit on va supposer que les leçons sont des cours de langue.

Au départ le concepteur a placé les données sur les leçons dans un unique fichier CSV dont voici un extrait du contenu sous forme de tableau.

idProf	professeur	idEtudiant	étudiant	langue	date
1	Luca	2	Philippe	italien	14 mars
2	Philippe	3	Lydia	français	15 mars
1	Luca	3	Lydia	italien	20 mars
2	Philippe	3	Lydia	français	2 avril
1	Luca	3	Lydia	italien	6 avril
3	Lydia	2	Philippe	anglais	12 avril
3	Lydia	2	Philippe	allemand	13 avril

On va chercher à faire mieux avec une base relationnelle.

Conception du schéma (7 pts)

- Pour chacune des DFs suivantes, indiquez si elle est respectée ou pas *dans le tableau précédent*. Si elle ne l'est pas, expliquez pourquoi.

- $professeur \rightarrow langue$
- $professeur, date \rightarrow etudiant$
- $etudiant \rightarrow professeur$
- $etudiant, langue \rightarrow professeur$

Correction

- $professeur \rightarrow langue$: non, car on voit que Lydia enseigne l'anglais et l'allemand
- $professeur, date \rightarrow etudiant$: oui, pas de contre-exemple dans le tableau (mais on en trouverait sans doute en continuant à accumuler les exemples)
- $etudiant \rightarrow professeur$, non, Philippe suit des cours avec Luca et Lydia
- $etudiant, langue \rightarrow professeur$: oui, pour une langue donnée, chaque étudiant n'a qu'un seul professeur

- Finalement, après avoir étudié beaucoup plus d'exemples, on détermine les dépendances fonctionnelles suivantes: $idProf \rightarrow professeur$, $idEtudiant \rightarrow etudiant$ et $(idProf, idEtudiant, date) \rightarrow langue$.

- Quelle est la clé de la relation ?
- Quelle est la décomposition en 3FN ?

Correction

- Quelle est la clé de la relation ? C'est le triplet (*idProf, idEtudiant, date*)
- Quelle est la décomposition en 3FN ? On applique la normalisation et on trouve 3 relations : Professeur (id, nom), Etudiant (id, nom) et Cours(idProf, idEtudiant, date, langue). Il est clair qu'il est préférable de fusionner les tables Professeur et Etudiant.

Pour la suite de l'examen nous travaillons avec le schéma suivant.

- Personne(**id**, nom, codeLangueNatale)
- Langue (**code**, intitulé)
- Cours(**id**, idProfesseur, idEtudiant, codeLangue, date)
- Sur ce schéma, répondez aux questions suivantes :
 - Enumérez les clés étrangères
 - Donnez le schéma entité-association correspondant
 - Donnez les commandes `create table`
 - Une des relations correspond-elle à une réification ? Si oui indiquez ce que serait le schéma sans réification.

Correction

- Clés étrangères `codeLangueNatale`, `codeLangue`, `idProfesseur` et `idEtudiant` dans Cours.
 - Standard
 - Standard
 - Réification : la relation Cours peut être interprétée comme venant d'une relation ternaire (Prof, Etudiant, Date), ce qui donnerait une clé avec trois attributs sans réification (voir les DF ci-dessus). Après réification, on a simplifié la clé, mais il faut prendre garde à ce qu'un professeur ne donne pas deux cours au même moment (avec une clause `unique`).

SQL (7 pts)

Sur le schéma donné précédemment, exprimez en SQL les requêtes suivantes :

- Liste des cours de l'étudiant Philippe (on suppose qu'il n'y en a qu'un), en indiquant le nom du professeur et la date.

Correction

```
select prof.nom as nomProfesseur,
from Cours as c, Personne as p, Personne as prof
where c.idEtudiant = p.id
and c.idProfesseur = prof.id
and p.nom='Philippe'
```

- Les personnes qui sont professeur et étudiant

Correction

Il faut trouver un cours où la personne est étudiante, un autre où la personne est professeur. Le `distinct` est utile.

```
select distinct p.nom
from Cours as c1, Cours as c2, Personne as p
where c1.idEtudiant = p.id
and c2.idProfesseur = p.id
```

- Les professeurs qui donnent des cours dans une autre langue que leur langue natale

Correction

Il faut trouver (au moins) un cours d'une langue qui n'est pas celle du professeur.

```
select distinct p.nom
from Cours as c, Personne as p
where c.codeLangue != p.codeLangueNatale
and c.idProfesseur = p.id
```

- Les professeurs qui ne donnent jamais cours dans une autre langue que leur langue natale

Correction

Ici, il faut que l'ensemble des cours dans une autre langue que la langue natale du professeur soit vide.

```
select p.nom
from Personne as p
where not exists (select *
from Langue as l, Cours as c
where c.idProfesseur = p.id
and c.codeLangue != p.codeLangueNatale)
```

- Les personnes et les langues qu'elles n'apprennent pas

Correction

On combine toutes les personnes et toutes les langues, on ne garde que les paires pour lesquelles il n'existe pas de cours

```
select p.nom, l.intitulé
from Personne as p, Langue as l
where not exists (select*
from Cours as c
and c.idEtudiant = p.id
and c.codeLangue = l.code)
```

- Donner, pour chaque personne, le nombre de langues qu'elle apprend

Correction

Petite astuce: il faut compter le nombre de langues distinctes (n'était pas pénalisé)

```
select p.nom, count(distinct langue.intitulé) as langue
from Personne as p, Cours as c, Langue as l
where c.idEtudiant = p.id
and c.codeLangue = l.code
group by p.id, p.nom
```

- Donner les langues pour lesquelles on trouve au moins trois personnes dont c'est la langue natale.

Correction

```
select l.intitulé, count(*) as nbPersonnes
from Personne as p, Langue as l
where l.code = p.codeLangueNatale
group by l.code
having count(*) >= 3
```

Algèbre (2 pts)

Donnez les expressions algébriques pour les requêtes suivantes :

- Nom des personnes qui se donnent un cours à elles-mêmes..

- Les personnes (on se contentera de l'identifiant) qui n'étudient pas l'italien (langue dont le code est `it`)

Correction

- $\pi_{nom}(\sigma_{idProfesseur=idEtudiant \bowtie idProfesseur=id} Personne)$
- $\pi_{id}(Personne) - \pi_{idEtudiant}(\sigma_{langue='IT'}(Cours))$

Valeurs nulles, vues (2 pts)

- Créez une vue `Inscriptions` donnant, pour chaque personne, le code des langues apprises, ou `NULL` si aucune langue n'est apprise.
- En utilisant cette vue, quelle requête donne les personnes qui n'apprennent aucune langue ?

Correction

- Il faut une jointure externe

```
create view Inscription as
select p.nom, c.codeLangue
from Personne as p outer join Cours as c
where c.idEtudiant = p.id
```

- On peut interroger très simplement la vue

```
select *
from Inscription as i
where i.codeLangue is null
```

Procédures (2 pts)

Expliquez ce que signifie l'immuabilité d'un curseur.

Correction

Voir le cours.

Examen session 1, 27 janvier 2024

On veut implanter une application de réservation de places en crèche. Voici le schéma proposé :

- Crèche (**idCrèche**, nom, ville, capacité)
- Personne (**idPersonne**, prénom, nom, néeEn, ville, idPère, idMère)
- Demande (**idDemande**, idEnfant, idCrèche, année)
- Inscription (**idEnfant**, **année**, idCrèche, idPuéricultrice)

Modélisation (8 points)

- Etudiez soigneusement le schéma et identifiez toutes les clés étrangères qui font référence à la table `Personne` (1 pt)

Correction

Il y en a beaucoup : `idPère`, `idMère`, `idEnfant` et `idPuericultrice`.

- Donnez les commandes SQL de création des tables suivantes : `Crèche`, `Personne` et `Inscription`. Note : on peut ne pas connaître le père ou la mère d'un enfant. (2 pts)

Correction

Voir ci-dessous.

```
create table Personne
(idPersonne int not null,
prénom varchar not null,
nom varchar not null,
néeEn int not null,
```

```

ville varchar not null,
idPère int ,
idMère int ,
primary key (idPersonne),
foreign key (idPère) reference Personne(idPersonne),
foreign key (idMère) reference Personne(idPersonne)
)

create table Inscription
(idEnfant int not null,
année int not null,
idCrèche int not null,
idPuericultrice int not null,
primary key (idEnfant, année),
foreign key (idEnfant) reference Personne(idPersonne),
foreign key (idCrèche) reference Crèche(idCrèche),
foreign key (idPuericultrice) reference Personne(idPersonne)
)

```

- Voyez-vous une table résultant d'une réification ? Une table résultant d'une entité faible ? Lesquelles ? (1 pt)

Correction

La table Inscription provient d'une entité faible : on voit que son identifiant comprend l'identifiant d'un enfant, auquel elle est donc indissolublement liée. La table Demande résulte d'une réification.

- Donnez le schéma Entité/association correspondant à ce schéma relationnel (2 pts)
- Un enfant peut-il être inscrit à plusieurs crèches la même année ; un enfant peut-il demander à s'inscrire dans plusieurs crèches la même année ? (1 pt)

Correction

Non, pas d'inscription d'un enfant la même année car la clé est constituée de ces deux attributs. En revanche on peut *demande* une inscription dans plusieurs crèches pour une même année.

- Si on ajoutait la dépendance fonctionnelle $idPuericultrice \rightarrow idCrèche$, comment faudrait-il modifier le schéma pour qu'il reste en 3FN ? (1 pt)

Correction

Il faut d'abord réaliser que `idPuericultrice` est un alias pour `idPersonne`. Il faut donc représenter la nouvelle dépendance fonctionnelle dans le schéma en ajoutant la clé étrangère `idCrèche` dans la table

Personne (elle peut être à null pour toutes les personnes qui ne sont pas puéricultrices).

On peut alors retirer `idCrèche` de la table Inscription puisque la crèche est déterminée par la puéricultrice.

Requêtes SQL (7 points)

Les requêtes sont à exprimer en SQL.

- Quels sont les enfants inscrits dans une crèche à Paris (donnez le nom de l'enfant et le nom de la crèche) ?

Correction

```

select p.nom as 'nomEnfant', c.nom as 'nomCrèche'
from Crèche as c, Inscription as i, Personne as p
where c.idCrèche = i.idCrèche
and i.idEnfant=p.idPersonne
and c.ville='Paris'

```

- Quels sont les enfants demandant une crèche dans la ville où ils habitent (donner le nom, prénom et la ville) ?

Correction

```

select p.prénom, p.nom, p.ville
from Crèche as c, Demande as d, Personne as p
where c.idCrèche = d.idCrèche
and d.idEnfant=p.idPersonne
and c.ville=p.ville

```

- Pour chaque enfant donnez le prénom de l'enfant, le nom de son père, de sa mère et de sa puéricultrice.

Correction

```

select enfant.nom as 'nomEnfant', père.nom as 'nomPère',
mère.nom as 'nomMère', puericultrice.nom as 'nomPuericultrice'
from Inscription as i, Personne as enfant, Personne as père,
Personne as mère, Personne as puericultrice
where i.idEnfant = enfant.idPersonne
and enfant.idPère = père.idPersonne
and enfant.idMère = mère.idPersonne
and i.idPuericultrice = puericultrice.idPersonne

```

- Je veux donner la liste de tous les enfants nés en 2022, avec le nom de leur puericultrice s'ils sont inscrits à une crèche, et null sinon. Donnez la requête SQL.

Correction

Il faut penser dans ce cas à effectuer une jointure externe.

```

select e.nom as 'nomEnfant', p.nom as 'nomPuericultrice'
from Personne as e left outer join
(Inscription as i join Personne as p on i.idPuericultrice=p.idPersonne)
on i.idEnfant = e.idPersonne
where néeEn=2022

```

- Indiquez les demandes en cours pour l'année 2024 : celles pour lesquelles il existe une demande de l'enfant à une crèche, mais pas d'inscription. Donnez simplement l'identifiant de l'enfant et l'identifiant de la crèche.

Correction

```

select d.idEnfant, d.idCrèche
from Demande as d
where année = 2024
and not exists (select * from Inscription as i
                where d.idEnfant = i.idEnfant
                and d.idCrèche = i.idCrèche
                and année = 2024)

```

- Donnez la liste des enfants nés en 2022 et les crèches de leur ville qu'ils n'ont pas demandées (nom de l'enfant, nom de la crèche).

Correction

```

select p.nom as 'nomEnfant', c.nom as 'nomCrèche'
from Personne as p, Crèche as c
where p.néeEn=2022
and p.ville = c.ville
and not exists (select * from Demande as d
                where d.idEnfant = p.idPersonne
                and d.idCrèche = c.idCrèche)

```

- Quelles sont les crèches n'ayant pas rempli toutes leurs places en 2024 (autrement dit le nombre d'inscriptions est inférieur à la capacité)?

Aide : faites une première requête affichant le nombre d'inscriptions et la capacité. Puis ajoutez la clause indiquant que le premier doit être inférieur au second.

Correction

```

select c.*
from Crèche as c, Inscription as i

```

```

where c.idCrèche = i.idCrèche
and année = 2024
group by c.idCrèche
having count(*) < c.capacité

```

Algèbre (2 points)

Donnez l'expression algébrique pour les requêtes 1 et 5 de la section précédente.

Correction

Pour la première on procède en 3 étapes, en définissant d'abord R_1 et R_2

- $R_1 := \pi_{nomEnfant, idCreche, nomCreche}((\rho_{nom \rightarrow nomEnfant}(Personne) \bowtie_{idPersonne=idEnfant} Inscription))$
- $R_2 := \pi_{idCreche, nomCreche}(\sigma_{ville='Paris'}(Crèche))$
- $R_1 \bowtie_{idCreche=idCreche} R_2$
- $\pi_{idEnfant, idCreche}(\sigma_{annee=2024}(Demande)) - \pi_{idEnfant, idCreche}(\sigma_{annee=2024}(Crèche))$

Programmation (3 points)

On implante une procédure d'inscription qui effectue les étapes suivantes :

- On parcourt les demandes des enfants qui ne sont pas encore inscrits
- Pour chaque demande on vérifie qu'il reste de la place dans la crèche demandée
- Si oui, on crée le nuplet dans la table `Inscription`

```

Procédure inscription()
begin
    demandes_en_cours = (résultat requête 5)
    for each demande in demandes_en_cours do
        places_restantes = (résultat requête 7)
        si (places_restantes > 0)
            Insérer inscription
        end for
    end
end

```

Questions

- Est-ce que l'insertion d'une inscription change le contenu du curseur sur les demandes en cours ? Justifiez votre réponse.
- Combien d'inscriptions ce programme va-t-il engendrer pour un même enfant ?
- En déduisez-vous une raison pour que l'exécution de ce programme soulève un problème, et lequel ?
- Que proposez-vous pour résoudre ce problème éventuel ?

Correction

- Non le curseur est immuable : cf. cours
- Autant d'inscriptions que de demandes, ce qui ne semble pas normal
- La base ne permet de représenter qu'une seule inscription pour une année, donc le programme rejettera les insertions dès la seconde tentative.
- Il faut sortir de la boucle dès la première inscription.

Examen session 1, 12 juin 2025

On met en place un système d'information sur les structures administratives françaises: régions, départements, préfectures. Voici le schéma de la base sur laquelle nous allons travailler. Les clés primaires sont en gras, les clés étrangères ne sont pas indiquées.

- **Personne**(**idPersonne**, nom, prénom)
- **Région**(**codeRégion**, intitulé, préfecture, idPrésident)
- **Département** (**codeDpt**, nom, préfecture, population, codeRégion, idPrésident)
- **Voisins** (**codeDpt1**, **codeDpt2**)

Dans les tables `Région` et `Département`, `préfecture` désigne la ville siège de la préfecture de région ou de département. Strasbourg est à la fois préfecture de la région *Grand Est* et du département du Bas-Rhin. Dans la même région, Colmar est préfecture du département du Haut-Rhin. Régions et départements sont présidés par une personne.

Deux départements sont voisins s'ils ont une frontière commune.

Compréhension du schéma (6 pts)

- Donner les commandes de création de table pour `Département` et `Voisins`. On supposera que `Personne` et `Région` ont déjà été créées.

Correction

- Standard. Voici pour la table `Voisins`:

```
create table Voisins
(
  idDpt1 int not null,
  idDpt2 int not null
  primary key (idDpt1, idDpt2),
  foreign key (idDpt1) references Département (idDpt),
  foreign key (idDpt2) references Département (idDpt)
)
```

- On ajoute la commande suivante au schéma de la table `Voisins`:

```
check (codeDpt1 < codeDpt2)
```

Quelle est à votre avis la motivation de cette contrainte?

Correction

Il existe une symétrie dans la notion de voisin: si A est voisin avec B, B est voisin avec A. Pour éviter le doublon de représentation (`voisin(A, B)` et `voisin(B,A)`), on décide de n'autoriser que le premier, d'où la clause `check`.

- Donner le schéma entité-association correspondant à ce schéma relationnel.

Correction

Standard

- Existe-t-il une association plusieurs-plusieurs dans ce schéma? Si oui vous paraîtrait-il judicieux de la réifier? Justifiez votre réponse.

Correction

`Voisins` provient d'une association plusieurs-plusieurs. La contrainte sur la clé implique qu'un département ne peut pas être deux fois le voisin d'un autre. C'est une contrainte tout à fait raisonnable, voire indispensable. La réification est donc inutile.

- Une même personne peut-elle présider plusieurs départements? Peut-elle présider un département et une région?

Correction

Réponse oui aux deux questions: aucune contrainte de clés n'empêche ces situations.

- Notons `D` le département, `P` la personne, `ND` le nom du département et `NP` le nom de la personne. Le schéma nous donne les dépendances fonctionnelles suivantes: `D -> P`, `ND -> P` et `P -> NP`.

On ajoute la dépendance `P -> D`. Comment l'interpréter et que devient le schéma relationnel?

Correction

La dépendance additionnelle indique qu'une personne ne peut diriger qu'un seul département. L'identifiant d'une personne pourrait donc servir à identifier un département. Ce ne serait cependant pas un très bon choix car la personne

dirigeant un département est amenée à changer régulièrement. En pratique, mieux vont donc déclarer dans le schéma une contrainte d'unicité sur l'identifiant `idPrésident` dans la table `Département` (c'est également une clé étrangère).

Dans toute la suite de l'examen, on travaille sur le schéma donné dans l'énoncé, et en supposant que la contrainte sur l'ordre des codes de département est respectée dans la table `Voisins` (donc, `codeDpt1 < codeDpt2`).

Algèbre et vues (4 pts)

On définit algébriquement les relations suivantes:

- $V_1 := Voisins$
- $V_2 := \rho_{codeDpt1 \rightarrow codeDpt2, codeDpt2 \rightarrow codeDpt1}(Voisins)$
- $V_3 := V_1 \cup V_2$

Répondez aux questions suivantes:

- Quel est le résultat de $V_1 \cap V_2$?
- Comment caractériseriez-vous le contenu de V_3 ?
- Donnez la commande SQL de création de la vue V_3 .
- Soit le département de code d_1 . Quelle requête algébrique sur V_3 donne les codes de tous ses voisins (expliquer).

Correction

V_1 est donc un synonyme de la table `Voisins`, et V_2 a le même contenu que V_1 mais l'ordre des clés est inversé. On ne trouve dans V_2 que les paires de département (d_1, d_2) tels que $d_1 > d_2$. D'où la réponse aux questions:

- $V_1 \cap V_2$ est vide
- V_3 contient toutes les paires de départements voisins (d_1, d_2) et (d_2, d_1) . Autrement dit, chaque relation de voisinage entre deux départements est représentée dans les deux sens.
- On peut donc créer la vue suivante qui pourra nous simplifier les requêtes SQL par la suite.

```
create view V3 as
select * from Voisins
union
select codeDpt2 as codeDpt1, codeDpt1 as codeDpt2
from Voisins
```

- Avec V_3 , on n'a plus besoin de se soucier de l'ordre des codes de département. On peut donc écrire soit:

$$\pi_{codeDpt2}(\sigma_{codeDpt1=d_1}(V_3))$$

soit

$$\pi_{codeDpt1}(\sigma_{codeDpt2=d_1}(V_3))$$

qui donneront le même résultat.

SQL (7 pts)

Exprimez en SQL les requêtes suivantes:

- Qui préside (prénom, nom) la région contenant le département "Cantal"?
- Quelles villes sont à la fois préfecture de région et d'un département de plus de 100000 habitants (donner le nom de la ville, du département, et l'intitulé de la région).
- Quels sont les départements voisins du Cantal ? (*Aide*: et si vous utilisiez la vue V_3 définie précédemment?)
- Quels départements d'une même région sont voisins l'un de l'autre? Donnez l'intitulé de la région et les noms des deux départements. Attention à bien prendre en

compte la contrainte sur les clés dans Voisins

- Quelles régions n'ont pas de département?
- Quelles personnes ne président ni région ni département?
- Donner, pour chaque région, le nombre de département et sa population totale (obtenue par cumul de celle des départements)
- Dans la requête précédente, que se passe-t-il si une région n'a pas de département? Comment réussir à afficher le nom de la région avec la valeur 0 pour le nombre de départements dans ce cas?

Correction

```
select prénom, nom
from Personne as p, Région as r, Département as d
where d.nom='Cantal'
and d.codeRégion = r.codeRégion
and r.idPrésident = p.idPersonne
```

```
select r.préfecture as ville, r.intitulé, d.noms
from Région as r, Département as d
where r.préfecture = d.préfecture
and d.population > 10000
```

Il est plus simple d'utiliser la vue V3.

```
select d2.nom as voisinCantal
from Département as d1, Département as d2, V3 as v
where d1.nom='Cantal'
/* On sait que dans V3, on trouve chaque voisinage
   représenté dans les deux sens */
and v.codeDpt1=d1.codeDpt
and v.codeDpt2=d2.codeDpt
```

Sinon, solution SQL complète avec un "ou" logique. Plus compliqué...

```
select d2.nom as voisinCantal
from Département as d1, Département as d2, Voisins as v
where d1.nom='Cantal'
and (
    /* Cas d'un voisin de code supérieur au Cantal */
    (d1.codeDpt < d2.codeDpt and v.codeDpt1 = d1.codeDpt1 and v.codeDpt2=d2.codeDpt)
    or
    /* Cas d'un voisin de code inférieur au Cantal */
    (d1.codeDpt > d2.codeDpt and v.codeDpt1 = d1.codeDpt2 and v.codeDpt2=d1.codeDpt)
)
```

```
select r.intitulé, d1.nom as nomDpt1, d2.nom as nomDpt2
from Région as r, Département as d1, Département as d2, Voisins as v
where r.codeRégion = d1.codeRégion
and r.codeRégion = d2.codeRégion
/* Pour prendre la relation 'voisins dans le bon sens' */
and d1.codeDpt < d2.codeDpt
and v.codeDpt1=d1.codeDpt
and v.codeDpt2=d2.codeDpt
```

```
select *
from Région as r
where not exists (
    select* from Département as d
    where r.codeRégion = d1.codeRégion)
```

```
select *
from Personne as p
where not exists (
    select * from Département as d
    where d.idPrésident = p.idPrésident)
and not exists (
    select * from Région as d
    where r.idPrésident = p.idPrésident)
```

```
select r.intitulé, count(d.codeDpt), sum(population) as population
from Région as r, Département as d
where r.codeRégion = d1.codeRégion
group by r.codeRégion, r.intitulé
```

Si on veut prendre en compte les régions sans département, il faut faire une jointure externe

```
select r.intitulé, count(d.codeDpt), sum(population) as population
from Région as r left join Département as d on r.codeRégion = d1.codeRégion
group by r.codeRégion
```

Les régions sans département apparaissent dans le résultat de la jointure avec une valeur à null pour la population, et le count appliqué à null renvoie 0 (tandis que le sum appliqué à null renvoie null).

Modèle relationnel (3 pts)

Soit les attributs TGV_R avec les dépendances fonctionnelles suivantes: $TG \rightarrow E$; $R \rightarrow G$; $E \rightarrow G$.

- Quelle est la clé?
- Quel est le résultat de l'algorithme de normalisation?
- Ce résultat est-il en troisième forme normale?

Correction

Tous les attributs qui n'apparaissent pas à droite sont nécessairement dans la clé, donc T, V et R. On vérifie aisément que TVR est une clé (la seule).

On a donc les relations (TGE) et (RG), auxquelles il faut ajouter la clé (TVR) (algorithme de normalisation).

Le résultat est en troisième forme normale, si l'on admet une petite extension: dans la relation TGE, la DF $E \rightarrow G$ n'a pas la clé pour partie gauche. Mais on ne peut pas décomposer plus sans perdre d'information. C'est un cas (très rare en pratique) où il faut admettre une définition de la 3FN un peu plus compliquée que celle donnée en cours.

[PRÉCÉDENT](#) | [INDEX](#)

[Annales des examens](#)

© Copyright 2002-2025, Philippe Rigaux. Created using [Sphinx](#) 4